A Multi-Expert Agentic Pipeline for Peptide Generation

Fitger Nathan, Iliescu Eduard, Macdonald Arthur, Roudeau Niels
Université de Bordeaux, Master 2, 2026
Supervisors: Akka Zemmari, Enzo Eddebbarh, Johan Samot

2026

Abstract

We present a proof-of-concept multi-expert agentic pipeline for *in silico* peptide generation [**?**]. A central orchestrator coordinates three specialized agents: an AI Designer that proposes candidates, a Chemistry Constraints agent that enforces proxy feasibility rules (length, net charge, hydrophobicity, and forbidden motifs), and a Biology Function/Risk agent that critiques expected activity and toxicity-related risks [**?**].

The pipeline iterates through a propose → filter → score → revise loop, with round-level logs for traceability. Evaluation is fully quantitative and reports validity, average and Top-K score, diversity, and constraint pass rates against a non-agentic baseline. Results show that iterative multi-agent coordination improves both constraint satisfaction and top-ranked candidate quality over single-pass generation [**?**, **?**, **?**].

Keywords: Multi-Agent Systems, Peptide Generation, In Silico Drug Design, Variational Autoencoder, Conditional Variational Autoencoder, Evolutionary Scale Modeling.

1 Introduction

Peptides, and especially antimicrobial peptides (AMPs), are promising candidates for therapeutic, agricultural, and industrial applications [**?**]. Their *in silico* design is typically formulated as an iterative optimization process: generate candidate sequences, apply formal constraints, and evaluate each candidate with proxy scores for activity, toxicity risk, and stability [**?**, **?**]. While recent machine learning methods have accelerated sequence design, balancing multiple and sometimes conflicting objectives remains difficult in practice.

This project addresses that challenge through an agentic proof of concept in which specialized software agents cooperate in a closed optimization loop. The pipeline includes an AI Designer agent that proposes candidates, a Chemistry Constraints agent that filters feasibility using proxy physicochemical rules, and a Biology Function/Risk agent that critiques expected function and risk. An orchestrator manages the iterative cycle (propose → filter → score → revise), applies stopping criteria, and stores logs for traceability.

The work is centered on AI aspects (modeling, orchestration, reasoning, and quantitative evaluation) and assumes no prior expertise in wet-lab biology or chemistry. Evaluation is explicit and reproducible: we measure validity, average and Top-K score, diversity, pass rates for constraints, and score evolution across rounds, then compare against a non-agentic baseline. This framing allows us to test whether multi-expert coordination provides measurable gains over single-pass generation .

The remainder of this paper is structured as follows: Section **??** provides a brief overview of the project context and related work. Section **??** details the architecture of our multi-agent pipeline and the specific roles of the AI, Chemistry, and Biology agents. Section **??** outlines our experimental setup and quantitative evaluation metrics. Section **??** presents our findings, including baseline comparisons and ablation studies. Finally, Section **??** concludes the report and discusses potential future work.

2 Context and Related Work

2.1 Evolution of In Silico Peptide Generation

The computational identification and design of therapeutic peptides have evolved from rule-based filtering to data-driven learning pipelines [**?**]. Early *in silico* methods relied on handcrafted descriptors and QSAR-style modeling, using aggregate physicochemical indicators (e.g., charge, hydrophobicity, or sequence length) as proxies for feasibility and activity [**?**]. These descriptors are still widely used as practical constraints, but current workflows increasingly combine them with learned sequence representations.

In recent years, generative and predictive models have enabled large-scale exploration of peptide sequence space. Broadly, modern systems map discrete sequences into continuous representations, generate candidate peptides under

configurable objectives, and score them with learned or heuristic predictors. This shift has improved throughput and candidate quality, but also highlighted a persistent challenge: no single model reliably satisfies multiple objectives at once (activity, safety, synthesizability, stability), which motivates more structured optimization workflows.

2.2 The Shift to Agentic Workflows

Despite these advances, peptide design pipelines often remain fragmented: generation, constraint checking, and biological scoring are implemented as separate components with limited coordination [**?**]. As a result, one-shot generation frequently produces candidates that satisfy part of the objective but violate other key constraints.

Agentic workflows address this by organizing the process as an explicit iterative loop: propose candidates, evaluate them with specialized modules, revise, and repeat [**?**]. Rather than relying on a single monolithic predictor, this design decomposes the objective into complementary roles and makes trade-offs explicit at each iteration. For constrained peptide optimization, the essential benefit is the coordination mechanism itself: specialized agents, shared evaluation signals, and closed-loop refinement [**?**].

2.3 Project Scope and Alignment

This project is scoped as an AI systems Proof of Concept for iterative peptide design. Concretely, the expected deliverable is an end-to-end multi-expert pipeline with three specialized agents and one orchestrator, where each iteration performs candidate proposal, chemistry-oriented constraint filtering, biology-oriented critique, and revision. The scientific objective is not to claim new biological mechanisms, but to test whether explicit role decomposition and orchestration improve optimization behavior under practical proxy constraints.

The project was supervised by an interdisciplinary team (AI, chemistry, and biology), which helped keep the work aligned with domain expectations, informed the definition of proxy constraints and evaluation criteria, and supported critical review of the reported results.

The work remains strictly *in silico*: no wet-lab validation is assumed, and all scores are documented proxies. Alignment with the project brief is ensured through quantitative reporting on validity, average and Top-K quality, diversity, constraint pass rates, and score evolution across rounds, followed by comparison to at least one non-agentic baseline. In this framing, success is defined by measurable and reproducible gains from iterative coordination rather than by absolute biological performance claims.

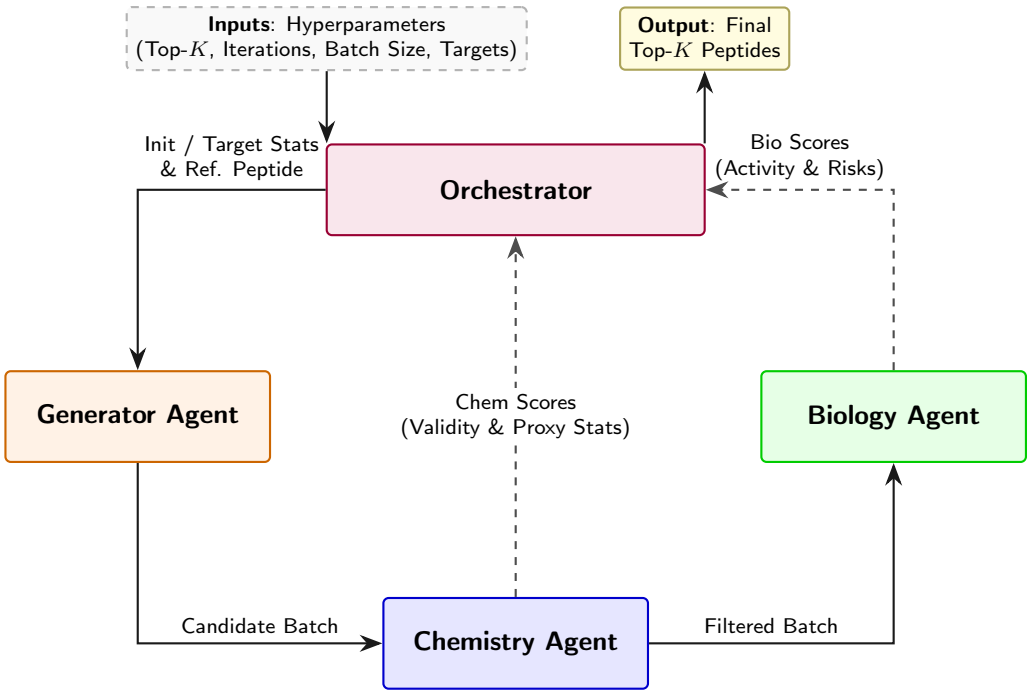

Figure 1: Agentic Pipeline Architecture. Solid lines denote the sequential flow of peptide candidate batches. Dashed lines indicate the feedback loops, where both the Chemistry and Biology agents return their respective proxy scores back to the Orchestrator. Color coding differentiates the specialized domains of the pipeline.

3 Multi-Agent System Architecture

The global pipeline is organized around three specialized agents coordinated by a central orchestrator. Figure **??** summarizes the end-to-end data flow and feedback loops in the iterative process.

3.1 The Orchestrator

The Orchestrator is the control layer that turns the three experts into a single iterative optimization system. Its role is to enforce a fixed sequence of operations:

1. Request a batch of candidates from the Generator (if not given when originally called)
2. Pass this batch through the Chemistry Agent to keep only chemically admissible peptides and associated proxy descriptors, and send the surviving candidates to the Biology Agent for biological scoring.
3. Based on these expert outputs, the Orchestrator ranks candidates, keeps the Top-K subset, and uses this subset as the seed for the next generation cycle.

In this way, each iteration combines exploration (new proposals) and exploitation (retaining best current candidates) under one consistent decision policy.

The stopping criterias are handled at the orchestration level rather than inside each expert module.
In practice, the loop can stop when :

- A maximum number of iterations is reached
- No valid candidates survive the chemistry gate
- The optimization objective is considered stable enough according to user-configured run settings (for example fixed Top-K, batch size, and iteration budget)

This implementation allows reproducible exepimentations : changing run behavior only requires updating orchestrator-level parameters, there is no need to change the experts. This agent also provides traceability by recording round-wise pipeline events and outcomes:

- Input settings
- Filtered populations
- Expert scores
- Selected candidates

This log-centric design makes the optimization path auditable from end to end: one can reconstruct why a peptide was retained or discarded at a given step, compare runs under different settings, and support post-hoc analysis of convergence behavior and failure cases.

3.2 Generator Agent

To generate novel peptide candidates, two generative architectures were implemented: a Variational Autoencoder (VAE) and a Conditional Variational Autoencoder (CVAE). Both models operate on peptide sequences encoded as fixed-length one-hot vectors, where each sequence position is represented over the 20 canonical amino acids. For a peptide of length L, the input dimensionality is therefore $20 \times L$. The objective of these models is to learn a structured latent representation of peptide sequences that can be sampled in order to generate new candidates. The VAE provides unconstrained generation based on the global training distribution, whereas the CVAE extends this framework by conditioning generation on physicochemical descriptors, enabling more directed design.

Variational Autoencoder

The VAE is a probabilistic generative model that learns to encode peptide sequences into a continuous latent space and decode latent samples back into

amino acid sequences. Given an input peptide representation x, the encoder predicts the parameters of a Gaussian latent distribution:

$$q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x)). \tag{1}$$

A latent vector z is then sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{2}$$

which allows gradients to propagate through the sampling step during training.

In the implemented architecture, the encoder is a multilayer perceptron composed of linear layers, layer normalization, and ReLU activations. The output is split into two vectors corresponding to the latent mean and log-variance. The decoder is another multilayer perceptron of symmetric structure that maps the latent vector back to the original sequence space.

Rather than reconstructing the sequence using a mean squared error loss, the decoder outputs raw logits for each amino acid at each sequence position. These logits are reshaped into categorical distributions over the 20 amino acids, and the reconstruction loss is computed using cross-entropy. This choice is more appropriate for peptide sequences, since amino acids are discrete categories rather than continuous values.

The training objective combines reconstruction quality and latent regularization:

$$\mathcal{L}(x) = \mathcal{L}_{recon}(x, \hat{x}) + \beta\, D_{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big), \tag{3}$$

where $p(z) = \mathcal{N}(0, I)$ is the prior distribution and D_{KL} denotes the Kullback–Leibler divergence. In practice, the KL term is progressively increased during training through KL annealing in order to stabilize optimization and reduce posterior collapse at early epochs.

The model is optimized with Adam, combined with cosine annealing of the learning rate and gradient clipping to improve training stability. During generation, latent vectors are sampled from the standard normal distribution and decoded into amino acid probability distributions. Sequence diversity is controlled through a temperature parameter applied before softmax sampling.

Conditional Variational Autoencoder

The CVAE extends the VAE by conditioning both encoding and decoding on an auxiliary vector y describing target peptide properties. Instead of modeling only the unconditional peptide distribution, the CVAE learns a conditional distribution:

$$q_\phi(z \mid x, y), \qquad p_\theta(x \mid z, y). \tag{4}$$

In the implemented model, the condition vector is concatenated with the peptide input before the encoder and concatenated again with the latent representation before the decoder. This architecture allows the model to generate peptides not only consistent with the training distribution, but also biased toward a specified physicochemical profile.

The condition vector is constructed from a fixed ordered list of physicochemical descriptors:

- length,
- molecular weight,
- net charge at pH 5.5,
- isoelectric point,
- hydrophobicity,
- hydrophobic moment,
- logP,
- Boman index,
- number of hydrogen bond donors,
- number of hydrogen bond acceptors,
- topological polar surface area (TPSA).

Each property is encoded using two values corresponding to a minimum and maximum target:

$$[min, max]. \tag{5}$$

If only a scalar value is provided, the same value is used for both entries. Missing values are set to zero. The resulting vector is either padded with zeros or truncated in order to match the predefined conditioning dimensionality.

This conditioning strategy enables the user to steer generation toward peptides with desired physicochemical characteristics. For example, the model can be guided toward sequences with a target range of charge, hydrophobicity, or molecular weight. In that sense, the CVAE is better suited than the VAE for directed peptide design.

Comparison Between VAE and CVAE

The VAE and CVAE serve related but distinct purposes. The VAE is an unconditional model: it learns the overall distribution of the training peptides and can sample novel sequences from this learned space. It is therefore useful for exploratory generation and for assessing whether the latent space captures relevant structural patterns in peptide sequences.

By contrast, the CVAE introduces control over the generation process. Because the model is explicitly conditioned on target descriptors, it can be used to bias sampling toward peptides exhibiting specific physicochemical properties. This makes the CVAE more appropriate for inverse design settings, where the goal is not merely to generate plausible peptides, but to generate candidates with a desired profile.

In summary, the VAE emphasizes diversity and distribution learning, whereas the CVAE emphasizes controllability and property-guided generation.

Physicochemical Conditioning

The physicochemical descriptors used by the CVAE can be computed from peptide sequences using the chemistry analysis module. These descriptors include molecular weight, charge, isoelectric point, hydrophobicity, hydrophobic moment, Boman index, and molecular descriptors derived from structure such as logP, hydrogen bond donor and acceptor counts, and TPSA.

This link between the chemistry module and the generative model is important, because it provides a principled way to translate biochemical objectives into numerical conditioning variables. As a result, the CVAE can be integrated into a broader peptide design pipeline in which candidate peptides are generated, analyzed, and filtered according to downstream criteria.

Limitations

Several limitations must be acknowledged. First, the current implementation assumes fixed-length peptide inputs, since the one-hot encoding and decoder output dimensionality depend directly on sequence length. As a result, the model does not naturally support variable-length peptides without additional preprocessing or architectural changes.

Second, the conditioning mechanism in the CVAE should be interpreted as a soft constraint rather than a hard constraint. The condition vector influences the probability distribution learned by the decoder, but it does not guarantee that every generated peptide will strictly satisfy the requested bounds. Consequently, an additional post-generation filtering step may still be necessary to enforce physicochemical thresholds.

Third, the usefulness of the CVAE depends strongly on the quality and consistency of the condition labels used during training. If the model is trained on sequences that are not paired with meaningful physicochemical targets, the conditioning mechanism will not learn a reliable relationship between conditions and generated sequences.

Finally, neither the VAE nor the CVAE directly predicts antimicrobial efficacy or biological activity. These models generate plausible peptide candidates from a learned sequence distribution, but downstream validation remains necessary to assess activity, selectivity, toxicity, and synthetic feasibility.

3.3 Chemistry Agent

The Chemistry Agent is the deterministic constraint-and-scoring stage of the pipeline. At a global level, it receives peptide sequences from the generator, evaluates them against user-defined physicochemical targets, and returns both a feasibility decision and a quantitative score used downstream by the orchestrator. The behavior of this agent is driven by a configurable target profile (ranges, target values, optional weights, and pH), which allows the same workflow to be reused under different experimental objectives.

Its workflow can be summarized through three complementary functions:

1. Sanitization : Generative models can hallucinate sequences that are syntactically correct but chemically invalid or unstable. The Chemistry Agent goes beyond simple string matching by attempting to instantiate a molecular graph for each sequence using the RDKit framework. This process involves a full sanitization step: checking valence rules, aromaticity, and atom consistency. Any sequence that fails to convert into a valid RDKit Mol object is immediately discarded as a chemical syntax error. This ensures that downstream agents only process realizable matter.

2. Multidimensional Property Profiling : To characterize developability, the agent computes a configurable set of descriptors from two complementary sources: peptide-centric properties (length, net charge at chosen pH, isoelectric point, hydrophobicity, cationicity) and molecule-centric properties (notably RDKit-based $\log P$). This mixed profile lets the orchestrator compare candidates in a unified physicochemical space.

3. Quantitative Scoring and Adaptive Filtering : The agent assigns each candidate a continuous score based on distance to target values, with optional per-property weighting. It then applies a two-step filtering policy: first keep peptides that satisfy all hard limits, then (if too few survive) refill the population with near-miss candidates that are closest to the target profile. This adaptive mechanism prevents early pipeline collapse while preserving optimization pressure.

The Chemist agent is the first called agent after the generator is used. It allows a first selection and analysis on the generated peptides list before sending them to the Biologist agent. Based on the strict limits provided by the user it sends back to the orchestrator scores and properties on the peptides. From an higher-level perspective, the returned data acts as a deterministic generated ground-truth wich can be trusted by the Biologist agent for its prediction.

3.4 Biology Agent (Function & Risk)

The Biology Agent evaluates the generated candidates and provides the biological signal used by the iterative loop. In our implementation, it is based on Meta/Facebook's Evolutionary Scale Modeling family (ESM-2) [**?**, **?**] and can be conditioned by context (for example an existing peptide reference or requested target properties).

At each iteration, the Biology Agent scores the current valid batch and returns values used by the orchestrator to rank candidates and keep the Top-K peptides for the next round. This makes it the core selection component on the biological side of the pipeline.

As a design choice, this score is treated as a relative proxy to guide search, not as a direct experimental measurement. The detailed scoring formulation is reported in Section **??**.

4 Experimental Setup

TODO: Describe the technical implementation (e.g., framework used, LLM endpoints).

4.1 Quantitative Scoring System

Biologist Agent Scoring

The objective is to rank a batch of peptides and identify which candidate is closest to the provided context (an existing peptide or requested properties represented as a reference embedding).

The implementation uses ESM-2 embeddings (default: `facebook/esm2_t12_35M_UR50D`) with attention-mask-aware mean pooling to produce one vector per peptide. We first tested cosine similarity between candidate and reference embeddings. In practice, cosine values were often too similar for short peptides in this protein-wide latent space, which reduced ranking contrast. We therefore switched to Euclidean distance, then normalized batch distances with a Z-score and mapped them to $[0, 1]$ with a sigmoid.

The resulting score is used for relative ranking inside each batch and for Top-K selection by the orchestrator. It should be interpreted as a proxy signal to guide the search process, not as an absolute biological activity measurement.

Chemist Agent Scoring

Crucially, to prevent pipeline starvation (where strict filters might discard the entire population early in the optimization process), the agent employs an adaptive thresholding mechanism. The workflow operates in two phases:

1. **Hard Constraint Check:** Candidates are first filtered against strict limits (e.g., range constraints on length or charge).

2. **Population Rescue:** If the survival rate falls below a predefined threshold (e.g., 50%), the agent relaxes the hard constraints. It then fills the remaining quota by selecting the top-ranking rejected candidates—those that, while slightly violating limits, are mathematically closest to the ideal TPP.

This strategy ensures a continuous flow of candidates to the Biology Agent while maximizing selective pressure towards developable molecules.

- **Solubility & Stability:** We calculate the Isoelectric Point (pI) and Net Charge at specific physiological pH levels (e.g., pH 5.5). These metrics are critical for predicting aggregation propensity and solubility in biological fluids.

- **Bioavailability & Permeability:** The generated profile includes the partition coefficient ($\log P$), Topological Polar Surface Area (TPSA), and

hydrophobicity indices. These values act as proxies for the peptide's ability to cross biological membranes and interact with hydrophobic pockets.

- **Structural Indices:** Metrics such as the Boman Index are computed to estimate potential protein-binding potential or antimicrobial activity.

Orchestrator-Level Scoring and Ranking

TODO: Describe how the orchestrator combines outputs from expert agents (Biologist + Chemist), applies weighting or filtering rules, and computes the final ranking used for Top-K selection.

4.2 Evaluation Metrics

TODO: Define the metrics you tracked: Validity (%), Diversity (distances/entropy/n-grams), and Constraint pass rates[cite: 25, 27, 28].

4.3 Baselines

The baseline is designed as a simpler non-agentic reference to test whether the proposed multi-expert iterative pipeline provides measurable gains. Concretely, we use a standalone Conditional Variational Autoencoder (CVAE) that directly generates peptide candidates from conditioning features, without chemistry/bio critique loops and without orchestrated revision rounds.

This baseline CVAE is trained on the dataset built with `database/get_data.py`. The extraction script queries DBAASP asynchronously and retains sequence-level and physicochemical fields used for conditioning (length, pH, molecular weight, logP, net charge, isoelectric point, hydrophobicity, and cationicity). In practice, many API entries contain missing values for one or more properties. The baseline data pipeline therefore substitutes absent fields with fixed defaults before standardization. This dataset is thus not optimal from a data-quality perspective, but for a proof-of-concept objective we considered it sufficient to establish a consistent comparison point.

From an implementation perspective, the baseline model is a GRU-based CVAE. The encoder embeds amino-acid tokens and maps sequence-condition pairs to latent Gaussian parameters; the decoder is autoregressive and conditioned on both latent code and property vector. Training uses cross-entropy reconstruction loss with KL regularization, Adam optimization, and a progressive KL weight (annealed β).

At inference time, candidate peptides are sampled from the latent prior under a target condition vector (scaled with the training `StandardScaler`), with temperature and top-k sampling to control exploration. This baseline serves only as the comparison anchor: unlike the full pipeline, it performs one-shot conditional generation and does not benefit from multi-agent feedback, constraint-aware iteration, or round-level refinement.

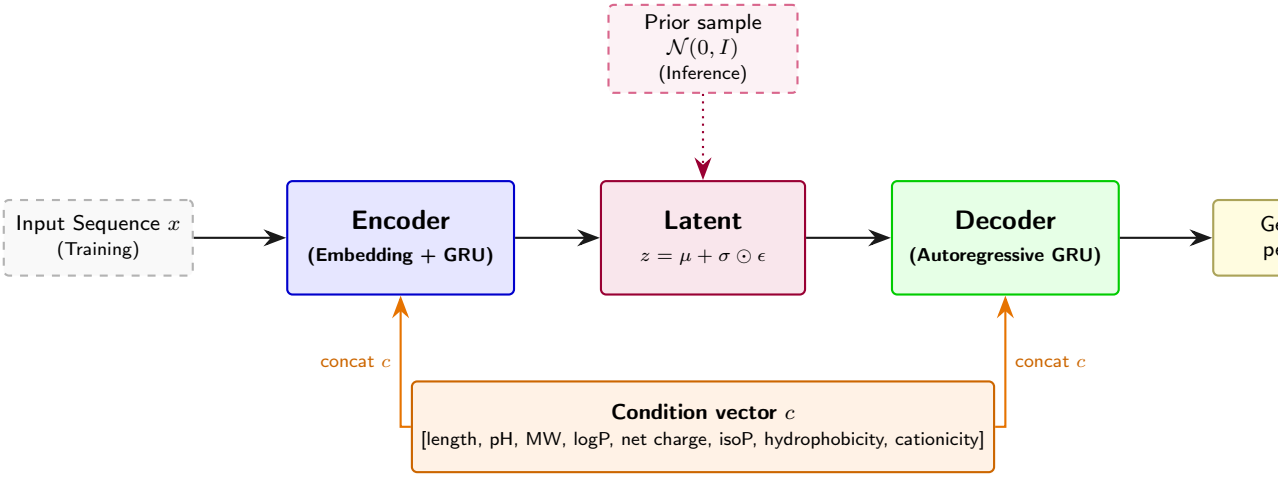

Figure 2: Baseline CVAE architecture used for comparison. During training, the model learns $q_\phi(z \mid x, c)$ and reconstructs sequences with an autoregressive decoder conditioned on z and c. During inference, peptides are generated by sampling $z \sim \mathcal{N}(0, I)$ and decoding once, without iterative multi-agent feedback.

5 Results and Analysis

5.1 Overall Performance and Top-K Gain

TODO: Present the final Top-K peptides. TODO: Show the measurable gain in average score and Top-K score compared to the non-agentic baseline[cite: 13, 26, 30]. Use a table.

5.2 Pipeline vs. Baseline Comparison

TODO: Add a direct comparison table between the full multi-agent pipeline and the standalone CVAE baseline (one-shot generation), using the same evaluation protocol. TODO: Report at least validity, average score, Top-K score, diversity, and chemistry-constraint pass rate for both systems. TODO: Add a short interpretation of the observed gap and indicate whether improvements are consistent across runs/seeds. TODO: Explicitly connect this comparison to the baseline definition in Section **??** and Figure **??**.

5.3 Trajectory of Improvement

TODO: Analyze the evolution of the scores across multiple iterative rounds. Include a graph plotting score vs. iteration number[cite: 29].

5.4 Ablation Studies

TODO: Present the results of running the pipeline with missing components to prove their worth. TODO: Compare the full pipeline vs. "Without Chemistry Agent", vs. "Without Biology Agent", vs. "Without Iteration"[cite: 42].

6 Discussion & Future Work

6.1 Chemist agent improvement

Trough the whole filtering and scoring process by the agent, the computed scores are obtained using a peptide sequence code (e.g "ACDEF", "AA"). The SMILES format is also very popular as its offer better topological information on the peptide, therefore we could be interested in gathering data from it, in addition to the current format. Unfortunately, our current lack of knowledge on the subject doesn't allow us to be certain about how this additional data could impact our scoring system and the overall pipeline.

6.2 Future Work: Peptide-Specific Biological Scoring

The main improvement is to replace the current proxy with a model trained (or fine-tuned) specifically on peptides. Ideally, this model should score or classify a peptide conditionally on a target (or target-related context), instead of relying on a general protein-space similarity signal.

The expected benefit is a more accurate and reliable biological signal for ranking candidates in the loop. A peptide-specialized model could also provide richer outputs than a single proxy score, for example clearer estimates of expected activity and additional information about risk or toxicity.

In short, our current biologist agent is a practical baseline for iterative optimization, while a target-conditioned peptide model is the natural next step to improve biological relevance and decision quality.

7 Conclusion

TODO: Summarize that the multi-expert agentic pipeline successfully demonstrates iterative improvement in peptide design. TODO: Mention that all code, scripts, and logs are provided in the Git repository[cite: 32, 33].

Bibliography

[1] F. Wan, F. Wong, J. J. Collins, and C. de la Fuente-Nunez, "Machine learning for antimicrobial peptide identification and design," *Nature Reviews Bioengineering*, Feb. 2024. [Online]. Available: https://doi.org/10.1038/s44222-024-00152-x

[2] A. Rives, J. Meier, T. Sercu, S. Goyal, Z. Lin, J. Liu, D. Guo, M. Ott, C. L. Zitnick, J. Ma, and R. Fergus, "Biological Structure and Function Emerge from Scaling Unsupervised Learning to 250 Million Protein Sequences," *Proceedings of the National Academy of Sciences*, vol. 118, no. 15, p. e2016239118, 2021.

[3] Z. Xi, et al., "The Rise and Potential of Large Language Model Based Agents: A Survey," *arXiv preprint arXiv:2309.07864*, 2023.

[4] T. Guo, et al., "Large Language Model based Multi-Agent Systems: A Survey of Progress and Challenges," *arXiv preprint arXiv:2401.03428*, 2024.

[5] D. A. Boiko, R. MacKnight, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[6] K. Huang, et al., "Biomni: A General-Purpose Biomedical AI Agent," *bioRxiv*, 2025.

[7] L. Wang *et al.*, "Machine learning for peptide design," *Nature Computational Science*, vol. 2, pp. 100–112, 2022.

[8] D. Veltri, U. Kamath, and A. Shehu, "Deep learning improves antimicrobial peptide recognition," *Bioinformatics*, vol. 34, no. 16, pp. 2740–2747, 2018.

[9] Facebook Research, "ESM: Evolutionary Scale Modeling," GitHub repository. [Online]. Available: `https://github.com/facebookresearch/esm`